UNIDADE I

Programação Orientada a Objetos I

Prof. Me. Tarcísio Peres

Conteúdo da disciplina

- Programação orientada a objetos (POO);
- Linguagem de programação C#;
- Orientação a objetos na prática: C# e .NET;
- Boas práticas de programação;
    - Tópicos especiais em C#;
    - Classes e algoritmos.

Conteúdo da Unidade I

- Entendendo classes e objetos;
- Os 4 pilares da POO;
- Histórico da POO;
- Criação de classes em C#;
- Interfaces e herança.

Programação estruturada

- Um programa é gerado a partir de um código-fonte, que é um conjunto de instruções escrito em determinada linguagem de programação.
- A primeira linguagem de programação surgiu em 1883 e foi criada pela primeira programadora do mundo, Ada Lovelace (Huskey; Huskey, 1980).
- Em 1949 surgiu uma linguagem de uso comercial: Assembly (Wilkes, 1949) e os códigos-fonte eram <u>estruturados</u> como uma lista de instruções a serem executadas sequencialmente.

Fonte: Imagem produzida pelo próprio autor com tecnologia DALL-E, uma ferramenta de inteligência artificial desenvolvida pela OpenAI.

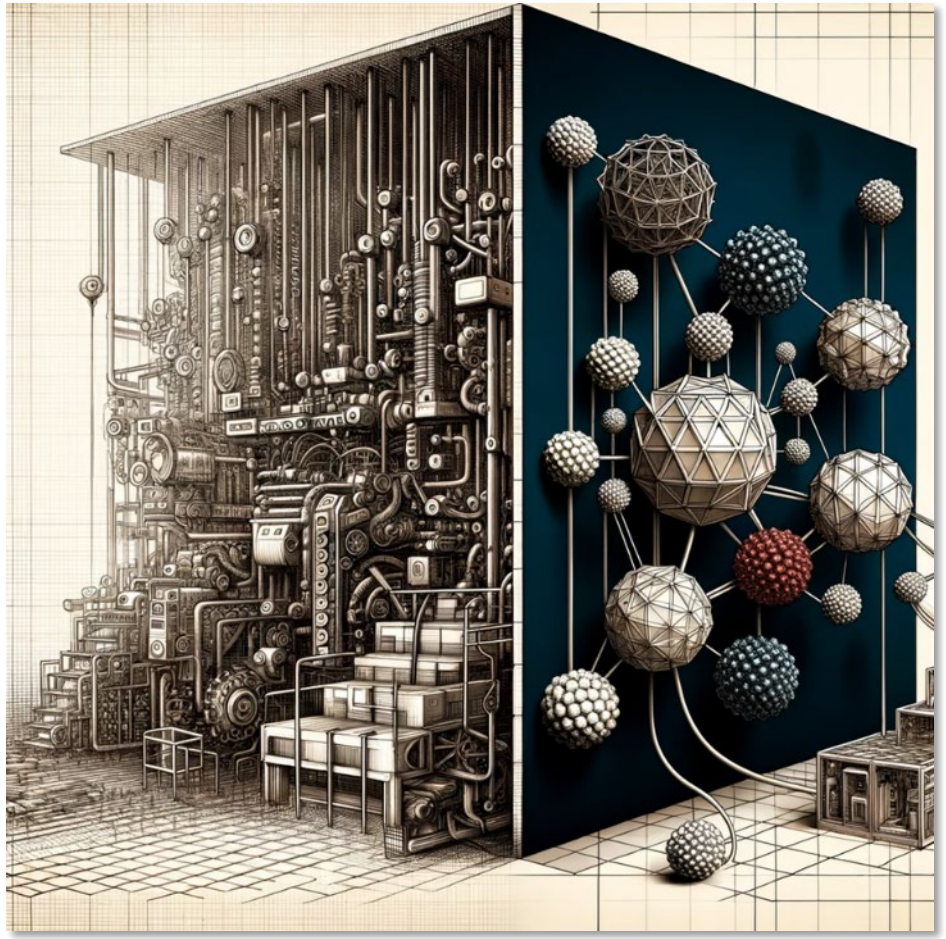
Programação estruturada

- O aumento do número de programas e, consequentemente, de programadores resultou no aumento proporcional de linhas de código e da sofisticação da lógica dos programas: os textos se tornavam cada vez maiores e mais complexos.
- O termo “programação macarrônica” se refere ao fenômeno de que alguns programas tinham um código-fonte “tão enrolado quanto um espaguete” (Steele, 1977).

Fonte: Imagem produzida pelo próprio autor com tecnologia DALL-E, uma ferramenta de inteligência artificial desenvolvida pela OpenAI.

Programação estruturada – Desafios e POO

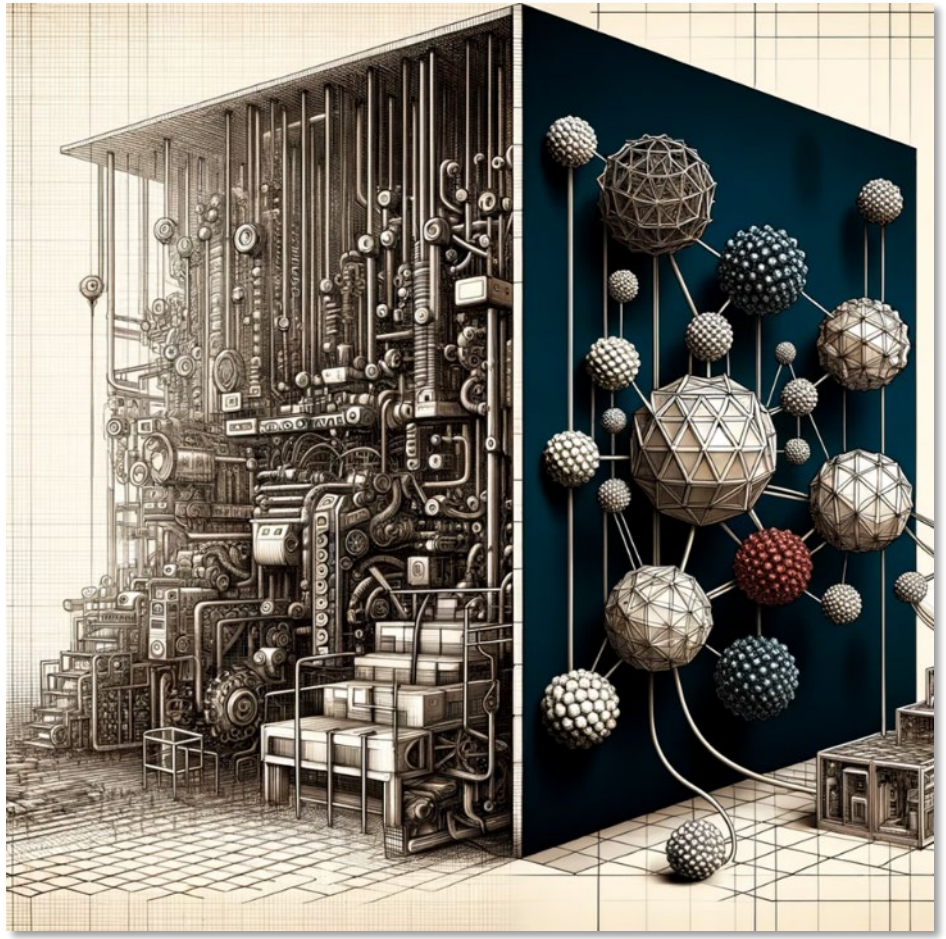
- O paradigma de programação estruturada não parecia se ajustar aos novos desafios de escala.
- Na segunda metade dos anos 1960, surgiu um novo paradigma com a proposta de lidar com a complexidade crescente dos programas de computador.
- Estratégia “dividir para conquistar”: dividir um problema grande em problemas menores, e dividir novamente os problemas menores em subproblemas, e assim sucessivamente. Resolvido cada subproblema individualmente, o problema original estará resolvido.



Fonte: Imagem produzida pelo próprio autor com tecnologia DALL-E, uma ferramenta de inteligência artificial desenvolvida pela OpenAI.

Programação estruturada – Desafios e POO

- A POO reduz o universo do problema em partes menores chamadas classes (ou objetos), e os programadores tornaram-se especialistas em modelar, criar e implementar tais objetos.
- A pioneira em implementar objetos e classes foi a Simula 67. Outros exemplos são Smalltalk, Emerald, Java, C++ e C#.
- Eu já sei programar: linguagem estruturada.



Fonte: Imagem produzida pelo próprio autor com tecnologia DALL-E, uma ferramenta de inteligência artificial desenvolvida pela OpenAI.

Programação orientada a objetos

- Classe: caneta.

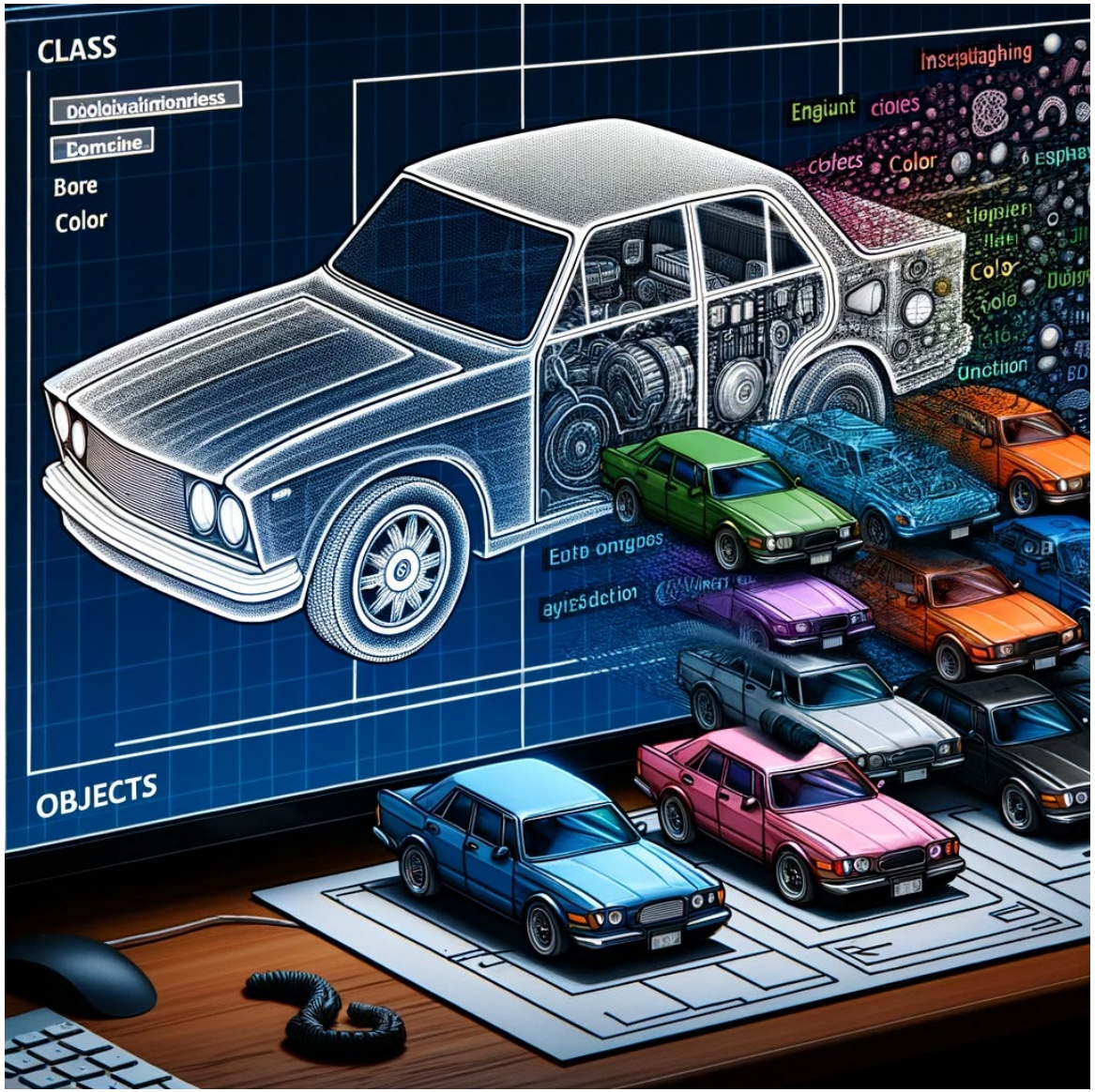
- Atributos (características) e métodos (comportamento).

- Objeto: uma caneta específica.

Fonte: Imagem produzida pelo próprio autor com tecnologia DALL-E, uma ferramenta de inteligência artificial desenvolvida pela OpenAI.

Programação orientada a objetos

- O objeto é um representante da classe.
- Podemos criar múltiplos objetos a partir de uma única classe.
- Cada objeto tem vida efêmera na memória volátil.



Fonte: Imagem produzida pelo próprio autor com tecnologia DALL-E, uma ferramenta de inteligência artificial desenvolvida pela OpenAI.

Os 4 pilares da POO

Abstração

Simplifica a representação de objetos do mundo real em software e permite que os desenvolvedores se concentrem nas características e comportamentos relevantes dos objetos, enquanto ignoram detalhes desnecessários.

Encapsulamento

Protege e controla o acesso aos dados e funcionalidades internas de um objeto, evitando modificações não autorizadas e minimizando a dependência entre diferentes partes do código.

Herança

Promove a reutilização de código e cria hierarquias de classes que compartilham características e comportamentos comuns, além de evitar a duplicação de código ao permitir que uma classe derivada herde atributos e métodos de uma classe-base.

Polimorfismo

Aumenta a flexibilidade e a extensibilidade do código ao permitir que um método seja usado de diferentes maneiras dependendo do contexto.

Interatividade

Qual alternativa indica uma diferença chave entre programação estruturada e POO?

a) Programação estruturada usa classes; POO usa funções.
b) Estruturada e POO diferem apenas na sintaxe.
c) Na programação estruturada, o foco está na sequência de ações a serem executadas, enquanto na POO, o foco está na definição de objetos que representam entidades ou conceitos do mundo real, com seus próprios dados e comportamentos.
d) A POO é mais antiga, baseada em linguagens de baixo nível.
e) A POO impede encapsulamento; a estruturada permite o encapsulamento completo.

Resposta

Qual alternativa indica uma diferença chave entre programação estruturada e POO?

a) Programação estruturada usa classes; POO usa funções.
b) Estruturada e POO diferem apenas na sintaxe.
c) Na programação estruturada, o foco está na sequência de ações a serem executadas, enquanto na POO, o foco está na definição de objetos que representam entidades ou conceitos do mundo real, com seus próprios dados e comportamentos.
d) A POO é mais antiga, baseada em linguagens de baixo nível.
e) A POO impede encapsulamento; a estruturada permite o encapsulamento completo.

4 Pilares – Abstração

- A Caneta abstrata

- class Caneta

```
public class Caneta
{
   private string cor;
   private bool tampada;

   public Caneta(string cor)
   {
      this.cor = cor;
      this.tampada = true; // Inicialmente, a caneta está tampada
   }

   public void Destampar()
   {
      if (tampada)
      {
           tampada = false;
           Console.WriteLine("A caneta foi destampada");
      }
      else
      {
         Console.WriteLine("A caneta já está destampada");
      }
   }
```

Fonte: autoria própria.

4 Pilares – Encapsulamento

- Público *vs* privado

- public class Caneta

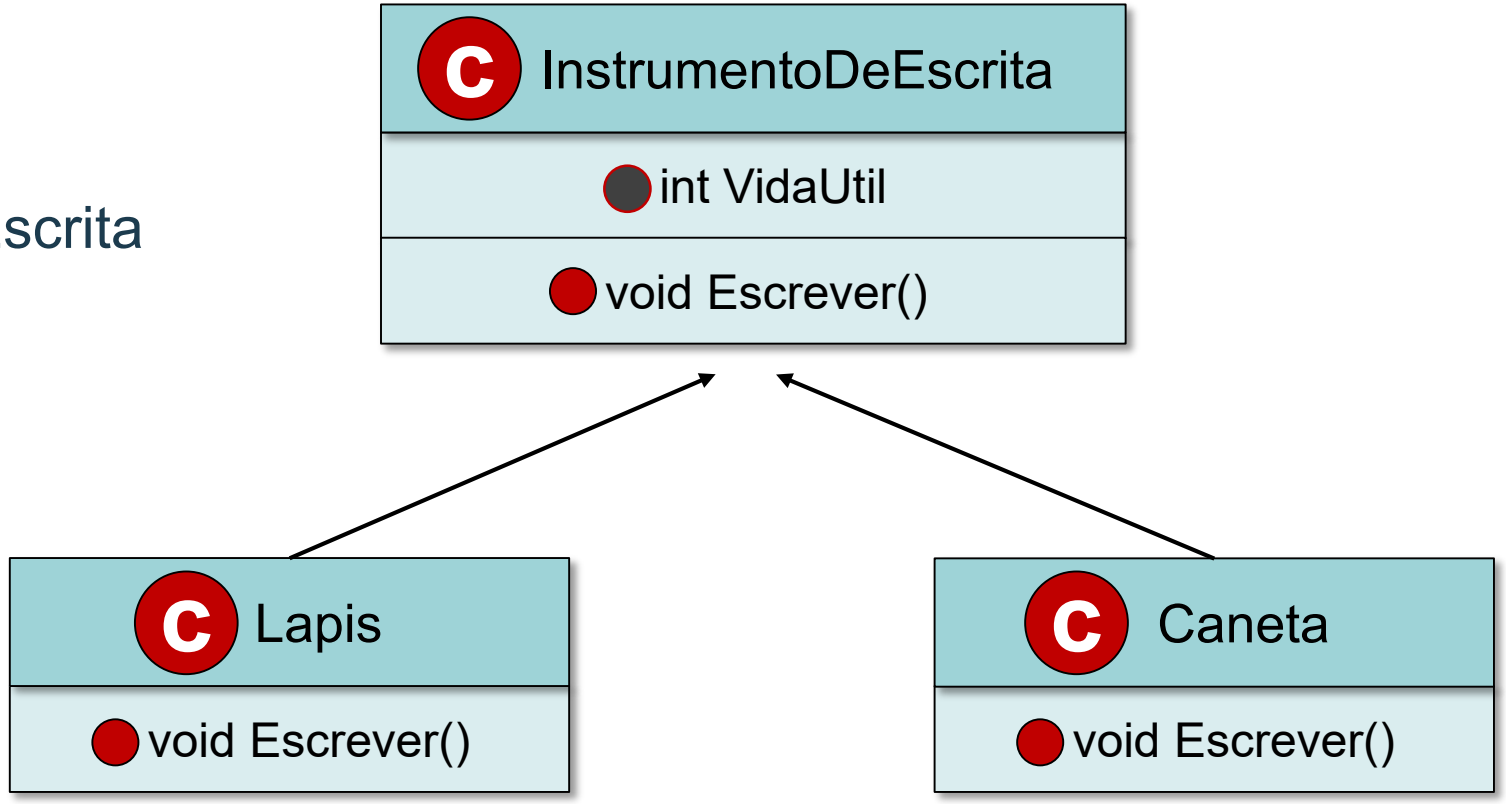
```
public class Caneta
{
   public string Cor { get; set; }          // Atributo público
   private int Carga { get; set; }          // Atributo privado

   public Caneta(string cor, int carga)     // Método Construtor
   { Cor = cor; Carga = carga; }

   public void Escrever(string texto)       // Método público para escrever
   {
      if (Carga > 0)
      {
         Console.WriteLine($"Escrevendo '{texto}' com uma caneta {Cor}.");
         Carga--;
      }
      else
      {
            Console.WriteLine("A caneta está sem carga.");
      }
   }

   public void Recarregar()                 // Método público para recarregar
   {
      Carga = 100;
      Console.WriteLine("Caneta recarregada.");
   }
```

Fonte: autoria própria.

Ativar
Acesse

4 Pilares – Herança

- Superclasse
- public class InstrumentoDeEscrita
- public class Lapis: InstrumentoDeEscrita



Fonte: autoria própria

4 Pilares – Herança

- Existem duas formas de herança: simples e múltipla.

- Algumas linguagens, como C# e Java, suportam apenas herança simples (uma classe pode herdar apenas de uma superclasse).

- Outras linguagens, como C++, permitem herança múltipla, na qual uma classe pode herdar de várias superclasses.

4 Pilares – Polimorfismo

Poli = muitos; Morph = formas

```
public abstract class InstrumentoDeEscrita {
    public abstract void Escrever();
}
public class Lapis : InstrumentoDeEscrita {
    public override void Escrever()  {
        Console.WriteLine("Escrevendo com o lápis!");
}
```

Fonte: Imagem produzida pelo próprio autor com tecnologia DALL-E, uma ferramenta de inteligência artificial desenvolvida pela OpenAI.

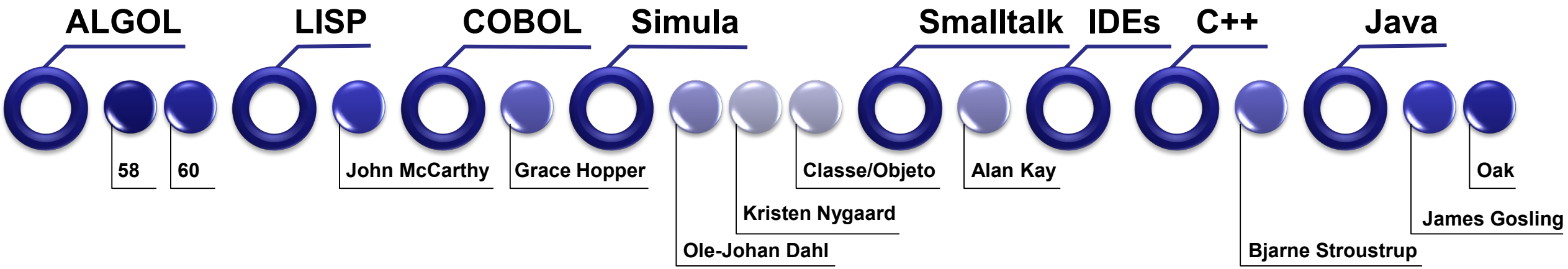
4 Pilares – Polimorfismo

- Subtipos (ou de inclusão): é o tipo mais comum em linguagens orientadas a objetos, como Java, C# e Python. Ocorre quando uma classe filha é tratada como instância de sua classe pai. Isto é, um objeto da subclasse pode ser usado em lugar de um objeto da superclasse.

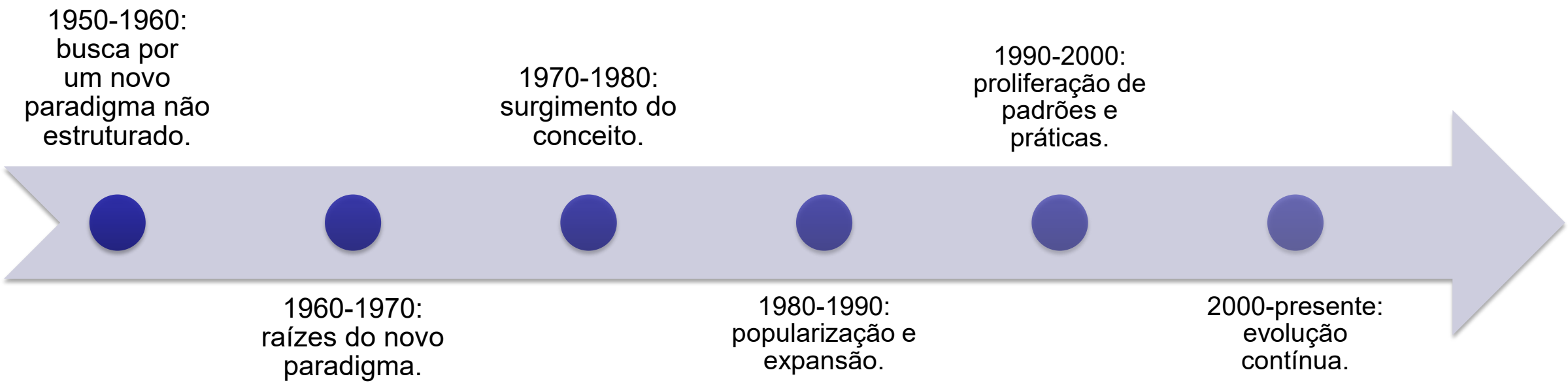
- Sobrecarga: refere-se à capacidade de várias funções terem o mesmo nome, mas com assinaturas diferentes. Isso significa que você pode ter múltiplos métodos com o mesmo nome na mesma classe, contanto que os parâmetros sejam diferentes.

- Paramétrico: geralmente associado a linguagens de programação genéricas, em que um tipo de dado é definido de maneira geral e depois especificado durante o uso.

Breve histórico – Linguagens



Breve histórico – POO



Programação C# – Declarando Classes

- modificador_de_acesso
  - *public*, *private*, *protected* e *internal*
- NomeClasse
- Atributos (variáveis de instância)
- Construtor (opcional)
- Métodos (funções)

```
1.  public class Pessoa
2.  {
3.     // Atributos
4.     public string Nome;
5.     public int Idade;
6.
7.     // Construtor
8.     public Pessoa(string nome, int idade)
9.     {
10.       Nome = nome;
11.       Idade = idade;
12.    }
13.
14.    // Método
15.    public void Apresentar()
16.    {
17.       Console.WriteLine($"Olá, meu nome é {Nome} e tenho {Idade}
anos.");
18.    }
19. }
```

Fonte: autoria própria

Programação C# – Alguns detalhes

- Identificadores válidos: o nome de classes deve começar com uma letra ou sublinhado (_) e pode conter letras, dígitos e sublinhados. Não pode começar com dígito.
- Case sensitive: os nomes de classe são case sensitive, ou seja, maiúsculas e minúsculas são distintas. Por exemplo, MinhaClasse e minhaclasse são nomes de classe diferentes.
- Palavras-chave: não se pode usar palavras-chave da linguagem C# como nome de classe. Por exemplo, você não pode nomear uma classe como *int*, *string*, *public* etc.
- Convenções de nomenclatura: embora não haja um limite estrito para o número de caracteres em um nome de classe, é prática comum seguir convenções de nomenclatura para manter o código legível.
    - A convenção de nomenclatura geralmente usada é o “PascalCase”, em que cada palavra no nome da classe começa com uma letra maiúscula e não contém espaços em caracteres especiais. Por exemplo, MinhaClasse, ClienteVIP etc.

Interatividade

Qual das seguintes afirmações melhor descreve a interação entre os quatro pilares da POO?

a) Encapsulamento e Abstração são independentes e não interagem com a Herança.
b) Encapsulamento é a base para Abstração, mas não tem relação com Polimorfismo.
c) Abstração refere-se exclusivamente à modelagem de dados, sem influenciar os conceitos de Encapsulamento, Herança e Polimorfismo.
d) Herança é um conceito que não se beneficia nem contribui para o Encapsulamento.
e) Encapsulamento protege os dados internos de uma classe, Abstração esconde a complexidade, Herança permite a reutilização de código, e Polimorfismo permite que objetos de diferentes classes sejam tratados como objetos de uma classe comum.

Resposta

Qual das seguintes afirmações melhor descreve a interação entre os quatro pilares da POO?

a) Encapsulamento e Abstração são independentes e não interagem com a Herança.
b) Encapsulamento é a base para Abstração, mas não tem relação com Polimorfismo.
c) Abstração refere-se exclusivamente à modelagem de dados, sem influenciar os conceitos de Encapsulamento, Herança e Polimorfismo.
d) Herança é um conceito que não se beneficia nem contribui para o Encapsulamento.
e) Encapsulamento protege os dados internos de uma classe, Abstração esconde a complexidade, Herança permite a reutilização de código, e Polimorfismo permite que objetos de diferentes classes sejam tratados como objetos de uma classe comum.

Programação C# – Atributos

```
public class Pessoa
{
    // Atributos
    public string Nome;
    public int Idade;
```

Fonte: autoria própria

- São as características ou propriedades dos objetos, representam os dados associados a uma classe e descrevem o estado de um objeto.
- São variáveis de instância dentro da classe e têm diferentes tipos de dados, como números, *strings* ou outros objetos.
- Encapsulam o estado de um objeto e podem ser acessados e modificados por métodos da classe.
- Por meio dos atributos, os objetos podem armazenar e representar informações relevantes para o problema que estão modelando, tornando a POO uma abordagem poderosa para organizar e manipular dados em programas de *software*.

Programação C# – Campos

```
public class Pessoa
{
    // Atributos
    public string Nome;
    public int Idade;
```

Fonte: autoria própria

- Campos (ou *fields* em inglês) são variáveis declaradas diretamente dentro de uma classe, estrutura ou enumeração, e não são encapsuladas por métodos *get* ou *set*.
- Eles representam o estado interno ou os dados armazenados por um objeto ou instância.
- Ao contrário de propriedades, campos são tipicamente usados para armazenamento interno, enquanto propriedades expõem dados ao exterior da classe de maneira controlada.
- Os termos “variável de instância” e “campo” são usados de maneira intercambiável. Ambos se referem a variáveis declaradas dentro de uma classe (ou estrutura) mas fora de qualquer método, construtor ou propriedade, e representam o estado ou os dados associados a uma instância específica da classe.

Programação C# – Propriedades

- São membros especiais que fornecem um mecanismo flexível para ler, escrever ou calcular o valor de campos privados, permitindo um controle mais detalhado sobre o acesso e a modificação dos valores do que simples campos.
- São definidas usando os métodos *get* e *set*, chamados acessadores.
- O acessador *get* retorna o valor da propriedade, enquanto o acessador *set* atribui um valor a ela.
- Podem ser de leitura, escrita ou ambas, e por meio delas é possível implementar lógicas adicionais, como validações, ao obter ou definir valores, facilitando a manutenção e melhorando a segurança do código.

```
public class Deidade
{
    public string Nome   {  get;   set;   }
    public string Dominio  {  get;  set;   }
}
```

Fonte: autoria própria

Programação C# – Constantes

- São variáveis cujos valores são definidos no momento da compilação e permanecem inalterados durante toda a execução do programa.
- São declaradas usando a palavra-chave const antes do tipo de dados; e uma vez atribuído um valor a uma constante, ele não pode ser modificado.
- Devido à sua natureza imutável, constantes devem ser inicializadas no momento da declaração.
- São úteis quando temos valores que não devem ser alterados, como o valor de pi (constante matemática) ou o número de dias na semana.

```
public class Semideus
{
    private string nomeSecreto  =  "Herói Desconhecido";
    public string Nome  {  get;  set;  }
    public const string Origem  =  "Monte Olimpo";
}
```

Fonte: autoria própria

Programação C# – Métodos

- Métodos são basicamente procedimentos ou funções associadas a uma classe ou objeto, têm um tipo de retorno.
- Um método pode ter parâmetros que permitem a passagem de informações para ele, permitindo que os métodos sejam mais flexíveis e reutilizáveis ao operar em diferentes conjuntos de dados.
- A assinatura de um método – que inclui seu nome e seus tipos de parâmetro (mas não seus tipos de retorno) – deve ser única dentro de uma classe. Os parâmetros são definidos na assinatura e, ao chamar o método, os argumentos devem coincidir com os parâmetros.

```
class Olimpo
{
  private int quantidadeDeuses = 12;

  public void InvocarZeus()
  {
    Console.WriteLine("Zeus foi invocado!");
    // Este método é void, então não tem um 'return' com um valor.
  }
```

Fonte: autoria própria

Programação C# – Passagem de parâmetros

- Por padrão, os parâmetros em C# são passados por valor. Isso significa que uma cópia do argumento é feita e passada para o método.

É possível passar um parâmetro por referência:

- ref: requer que a variável passada esteja inicializada antes de ser passada ao método.
- out: não exige que a variável passada esteja inicializada, mas requer que o método inicialize o parâmetro antes de terminar.

Fonte: Imagem produzida pelo próprio autor com tecnologia DALL-E, uma ferramenta de inteligência artificial desenvolvida pela OpenAI.

Programação C# – Métodos estáticos

- São membros da classe, em vez de serem associados a uma instância específica dela.
- Não é necessário ter uma instância de um objeto para chamar um método estático.
- São frequentemente usados para representar operações gerais para uma classe.
- Vantagens: eficiência (como eles não operam em dados de instância, não é necessário criar um objeto para usá-los) e conveniência (eles podem ser chamados diretamente de uma classe, o que pode ser mais intuitivo em certos contextos).
- Limitações: eles não podem acessar campos ou métodos de instância da classe e não podem ser sobrescritos em uma classe derivada.

```
public class DeusesGregos
{
   public static string DeidadeDoConceito(string conceito)
   {
```

```
string deusDoMar = DeusesGregos.DeidadeDoConceito("mar");
```

Fonte: autoria própria

Programação C# – Operador de acesso

- Operador de acesso (*dot operator* em inglês) é usado para acessar membros de um objeto ou tipo.

```
1.  public class HeroisGregos
2.  {
3.     public static bool RealizouFeito(string heroi, string feito)
4.     {
5.        if (heroi == "Hércules" && feito == "12 trabalhos")
6.             return true;
7.        if (heroi == "Perseu" && feito == "Matar Medusa")
8.             return true;
9.        // ... outros heróis e feitos ...
10.
11.       return false;
12.    }
13. }
14.  // ... Main
15. bool herculesFeito = HeroisGregos.RealizouFeito("Hércules", "12
trabalhos");
16. Console.WriteLine(herculesFeito);  // Isso imprimirá "true"
```

Fonte: autoria própria

Programação C# – O método Main

- Método estático é um método que pertence à própria classe, e não a uma instância específica de uma classe. Isso significa que você pode chamar um método estático diretamente no nome da classe, sem precisar criar um objeto dessa classe.

- O método estático Main é especial na linguagem C# por ser o ponto de entrada de um programa. Se compilarmos e executarmos um programa C#, o ambiente de execução .NET (como o .NET Core ou .NET Framework) carrega o assembly (o arquivo compilado) e procura pelo método Main como ponto de entrada. Uma vez encontrado, ele começa a execução a partir desse método.

```
    public static void Main(string[] args)
    {
        InstrumentoDeEscrita lapis = new Lapis();
        InstrumentoDeEscrita caneta = new Caneta();

        ExecutarEscrita(lapis); // Output: Escrevendo com o lápis!
        ExecutarEscrita(caneta); // Output: Escrevendo com a caneta!
    }
}
```

Fonte: autoria própria

Programação C# – Métodos de extensão

```
public static class ExtensaoDeus
{
   public static bool EhReiDoOlimpo(this Deus deus)
   {
      return deus.Nome == "Zeus";
   }
}
```

Fonte: autoria própria

- Permitem adicionar novos métodos a tipos existentes sem modificar o tipo original.
- São úteis para adicionar funcionalidades a classes ou estruturas que não podemos ou não desejamos modificar.
- Para criar um método de extensão, o método deve ser estático e definido em uma classe estática.
- O primeiro parâmetro do método deve usar a palavra-chave *this*, seguida pelo tipo que desejamos estender.

Programação C# – Parâmetros opcionais e nomeados

- Permitem especificar um valor-padrão para um parâmetro.
- Ao chamar o método, você pode omitir argumentos para quaisquer parâmetros opcionais.

```
public class Heroi
{
        public string Nome { get; set; }
        public void RealizarMissao { string missão = "Missão desconhecida" }
        {
                Console.WriteLine {$"{Nome} está realizando: {missão}"};
        }
}
```

```
1. Heroi heroi = new Heroi { Nome = "Hércules" };
2. heroi.RealizarMissao(); // imprimirá "Hércules está realizando:
   Missão desconhecida"
3. heroi.RealizarMissão("12 trabalhos"); // imprimirá "Hércules está
   realizando: 12 trabalhos"
```

```
1. public void RegistrarDeus { string nome, string dominio =
   "Desconhecido", bool ehOlimpiano = true)
2. {
3.         // Lógica do método
4. }
```

Fonte: autoria própria

Programação C# – Sobrecarga

- Permite múltiplos métodos com o mesmo nome na mesma classe, mas com diferentes listas de parâmetros.
- A sobrecarga de método é feita dentro da mesma classe (ou em classes derivadas, se a herança estiver envolvida).
- A distinção entre diferentes sobrecargas do mesmo método é baseada nos tipos, na ordem e no número de parâmetros.

```
// Exemplo básico:
Console.WriteLine("Olá, mundo!");

// Usando formatação:
string deus = "Zeus";
Console.WriteLine("O deus supremo é {0}.", deus);

// Usando interpolação de string (introduzida em C# 6):
Console.WriteLine($"O deus supremo é {deus}.");

// Escrevendo valores numéricos:
int idadeDeZeus = 3000;
Console.WriteLine($"Zeus tem {idadeDeZeus} anos.");
```

Fonte: autoria própria

Programação C# – Construtores

- É um método especial de uma classe ou estrutura em C# automaticamente invocado quando um objeto daquela classe é criado.

Seu principal objetivo é inicializar um objeto, e suas duas características básicas são:

- Ter o mesmo nome da classe;
- Não ter valor de retorno, nem mesmo *void*.

- Toda classe em C# tem um construtor, mesmo que não o declaremos explicitamente.
- Se você não fornecer um construtor para sua classe, o C# criará automaticamente um construtor-padrão sem parâmetros, que será chamado de construtor padrão implícito.
- Pode ter mais de um construtor por classe, inicializando o objeto de maneiras diferentes.

Programação C# – Desconstrutores

- Relaciona-se com o recurso introduzido na versão 7, chamado desconstrução, que permite decompor objetos em variáveis individuais.
- É particularmente útil com tipos de valor como tuplas, um conjunto ordenado de elementos.
- A desconstrução permite extrair múltiplos campos, propriedades ou outros valores de um objeto em muitas variáveis.
- Um objeto que suporta desconstrução deve ter um método Deconstruct que o desmonte.
- O método não retorna valores, mas usa parâmetros out para isso.

Programação C# – Destrutores

- Libera recursos ou realiza outras operações de limpeza antes que um objeto seja recolhido pelo coletor de lixo (*garbage collector*), característica fundamental da plataforma .NET.
- Gerencia a alocação e a liberação automática da memória para as aplicações – processo crucial para garantir a eficiência e estabilidade das aplicações.
- Só podem ser usados em classes. Têm o mesmo nome da classe, precedido pelo símbolo: ~
- Não podem ser herdados nem sobrecarregados e não têm parâmetros nem modificadores de acesso.
- Em C#, um programador raramente precisará escrever um destrutor explicitamente, uma vez que o coletor de lixo lida com a maioria das tarefas de limpeza automaticamente.

Interatividade

Qual das seguintes declarações sobre atributos, métodos e classes em C é a correta?

a) Métodos não podem ser sobrecarregados dentro da mesma classe.
b) Os atributos não têm nenhum impacto no comportamento dos métodos da classe.
c) Todas as classes herdam diretamente da classe Object, e é obrigatório sobrescrever todos os seus métodos, como ToString(), Equals() e GetHashCode(), em cada classe derivada.
d) Os métodos podem ser declarados com quaisquer modificadores de acesso (*public*, *private* etc.) em interfaces, assim como em classes.
e) C# permite a sobrecarga de métodos dentro da mesma classe e a herança de classes, com a possibilidade de sobrescrever métodos herdados.

Resposta

Qual das seguintes declarações sobre atributos, métodos e classes em C é a correta?

a) Métodos não podem ser sobrecarregados dentro da mesma classe.
b) Os atributos não têm nenhum impacto no comportamento dos métodos da classe.
c) Todas as classes herdam diretamente da classe Object, e é obrigatório sobrescrever todos os seus métodos, como ToString(), Equals() e GetHashCode(), em cada classe derivada.
d) Os métodos podem ser declarados com quaisquer modificadores de acesso (*public*, *private* etc.) em interfaces, assim como em classes.
e) C# permite a sobrecarga de métodos dentro da mesma classe e a herança de classes, com a possibilidade de sobrescrever métodos herdados.

Programação C# – Objetos

- A criação de objetos é peça fundamental na POO.

- A capacidade de criar e manipular objetos é o que dá vida às aplicações, permitindo que os desenvolvedores representem entidades, processos ou conceitos do mundo real dentro de um ambiente de software.

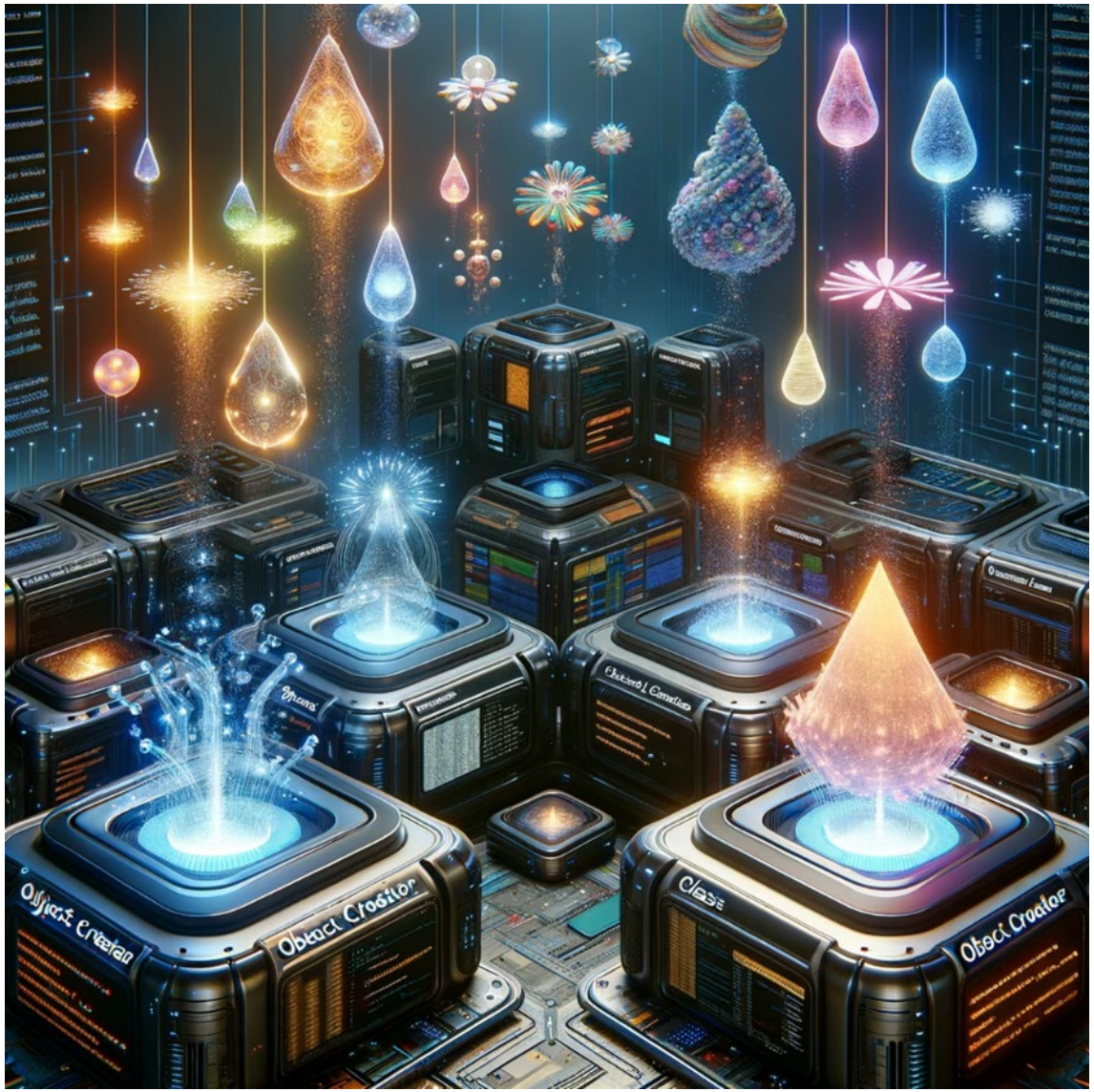
- Criar um objeto significa criar uma instância dessa classe, um processo conhecido como instanciação.

  - Ao se criar a instância de uma classe, reserva-se espaço na memória para essa instância, iniciada com quaisquer valores-padrão ou valores fornecidos.

Programação C# – Objetos

- Ao criar um objeto, não estamos apenas produzindo a cópia passiva de um modelo, mas trazendo à vida uma entidade com estado e comportamento.

- Esse objeto pode interagir com outros objetos, pode ser modificado e pode desencadear uma variedade de ações e comportamentos dentro do software.



Fonte: Imagem produzida pelo próprio autor com tecnologia DALL-E, uma ferramenta de inteligência artificial desenvolvida pela OpenAI.

Programação C# – Objetos

- Um objeto permanecerá na memória enquanto houver referência a ele.

- Se não houver mais referências apontando para esse objeto, ele se torna elegível para coleta de lixo (*garbage collection*).

- Não há limite fixo para o número de objetos que podemos criar em C#, mas na prática o número de objetos que podem ser criados é limitado pela quantidade de memória disponível no sistema.

  - Deve-se sempre estar ciente do consumo de memória, especialmente em aplicações que requerem alta performance ou que serão executadas em dispositivos com recursos limitados.

Programação C# – new

- Palavra-chave: *new*.
- zeus é o nome do objeto: referência em memória.
- Deus é a classe.
- Deus (XXX, YYY) é o método construtor da classe.
- zeus.Nome é o acessador.
- atena, afrodite e hades são objetos distintos.
- atena.ExibirPoderes()

```
public class MeusPrimeirosObjetos
{
   public static void Main(string[] args)
   {
      Deus zeus = new Deus("Zeus", "Ceu e Trovao");
      Console.WriteLine(zeus.Nome); // Saída: Zeus
      zeus.ExibirPoderes(); // Saída: Zeus é o(a) deus(a) do(a) Céu e
ão.
      Deus atena = new Deus("Atena", "Sabedoria");
      Deus afrodite = new Deus("Afrodite", "Amor");
      Deus hades = new Deus("Hades", "Submundo");
      atena.ExibirPoderes(); // Saída: Atena eh o(a) deus(a) do(a)
edoria.
   }
}
```

Fonte: autoria própria

Programação C# – this

- É usado para referir-se à instância atual da classe em que o método ou a propriedade está sendo executada.
- É útil para desambiguar membros da classe e parâmetros, especialmente quando têm o mesmo nome.
- *this* também pode ser usada dentro de um construtor para invocar outro construtor na mesma classe. Isso é útil se a classe tiver múltiplos construtores e quisermos evitar a repetição de código.

```
public class Heroi
{
   private string nomeDeus;

   public Heroi(string nomeDeus)
   {
      this.nomeDeus = nomeDeus;
   }
}
```

```
public class Deidade
{
   private string nome;
   private string dominio;

   public Deidade(string nome) : this(nome, "Desconhecido") { }

   public Deidade(string nome, string dominio)
   {
      this.nome = nome;
      this.domínio = dominio;
   }
}
```

Fonte: autoria própria

Programação C# – Classes estáticas

- É definida usando a palavra-chave *static*, e sua principal característica é não ser instanciada.
- Não podemos criar um objeto de classe estática usando o operador *new*.
- Todos os seus membros (campos, propriedades e métodos) devem ser obrigatoriamente estáticos.
- Para acessar um membro estático, usa-se o nome da classe, seguido pelo nome do membro.
- São frequentemente usadas como utilitários ou ferramentas.
- As vantagens de usar classes estáticas são o desempenho e a organização do código. Uma vez que não é necessário instanciar, o acesso a membros de uma classe estática pode ser ligeiramente mais rápido.

```
public static class OraculoDeDeltos
{
   public static string Profecia()
   {
      return "Um grande herói surgirá!";
   }
}
```

Fonte: autoria própria

Programação C# – Classes estáticas

- Uma das limitações de classes estáticas é que elas não podem ser herdadas por outras classes.

Há várias classes estáticas populares e amplamente utilizadas em C#, dentre elas:

- System.Math: fornece funções matemáticas básicas, e todos os seus membros são estáticos.
- System.Console: é predominantemente usada por seus membros estáticos para manipular a entrada e saída do console.
- System.Environment: fornece informações sobre o ambiente de execução atual e várias utilidades relacionadas ao sistema.

```
double raiz = Math.Sqrt(16);      // Calcula a raiz quadrada
double potencia = Math.Pow(2, 3);// Eleva 2 à potência de 3
double maior = Math.Max(5, 7);    // Retorna o maior valor entre 5 e 7
```

Fonte: autoria própria

Programação C# – Interface

- É um contrato que outras classes podem implementar.
- Define um conjunto de assinaturas de métodos, propriedades, eventos ou indexadores sem implementar a funcionalidade real desses membros (define o “o quê”, mas não o “como”).
- Uma classe só pode herdar de uma única classe-base, mas pode implementar múltiplas interfaces.
- Ajudam a separar definição e implementação, permitindo que o programador defina operações sem especificar como são realizadas.
- Permite que objetos de diferentes tipos sejam tratados como objetos de um tipo comum.

```
public interface IVoador
{
   void Voar();
}

public interface IAtacante
{
   void Atacar();
}
```

Fonte: autoria própria

Programação C# – Classe abstrata

- É semelhante a uma interface, mas permite fornecer alguma implementação-padrão para seus membros.
- Pode ter métodos com implementação completa, mas não pode ser instanciada diretamente.
- Outras classes devem herdar dessa classe abstrata e implementar métodos abstratos sem implementação.

  - A principal diferença entre classes abstratas e interfaces é que uma classe pode herdar apenas de uma classe abstrata (devido à restrição de herança única em C#), mas pode implementar várias interfaces.

Fonte: autoria própria

Programação C# – Classe abstrata

Critério	Interface	Classe abstrata
Definição	Contrato que define a assinatura de métodos, propriedades e outros membros sem implementá-los.	Pode ter tanto membros abstratos (sem implementação) quanto concretos (com implementação).
Instanciação	Não é possível instanciar diretamente.	Não é possível instanciar diretamente.
Herança	Uma classe pode implementar várias interfaces.	Uma classe só pode herdar de uma classe abstrata.
Membro	Só podem conter declarações.	Pode ter declarações e implementações.

Fonte: Livro-texto

Fonte: autoria própria

Programação C# – Hierarquia de classes

- É uma estrutura crucial que ajuda os desenvolvedores a organizar e estruturar seu código de maneira lógica e intuitiva.
- Refere-se à maneira como as classes são organizadas umas em relação às outras, normalmente para representar algum tipo de relação “é um tipo de”.
- É possível identificar classes mais gerais e mais específicas, permitindo reutilizar códigos e facilitando seu entendimento e manutenção.
- No centro da hierarquia de classes em C#, temos as classes-base e as classes derivadas. Uma classe-base pode servir como ponto de partida para outras classes; estas, por sua vez, que derivam da classe-base, são chamadas classes derivadas.
    - Em C# e no .NET Framework (e .NET Core/.NET 5+), há uma hierarquia inerente à estrutura das classes que formam o *framework*.
    - A classe-base para todas as classes em .NET é System.Object, e toda classe em C#, explicitamente herdada de outra classe ou não, deriva de System.Object no final da hierarquia.

Programação C# – Herança

- A herança simples é codificada utilizando o operador dois-pontos (:).
- A classe derivada herda membros (métodos, propriedades etc.) da classe-base.
- Ao criar uma classe-base com características e comportamentos comuns, evita-se a duplicação de código nas classes derivadas.
- Em termos de acessibilidade, uma classe derivada tem acesso a todos os membros *public* (acessíveis por qualquer código) e *protected* (acessíveis pela classe-base e por qualquer classe derivada) da classe-base. Ela não tem acesso a membros private (acessível apenas pela classe que o declara) da classe-base.
    - *Overriding* é a capacidade de uma classe derivada fornecer implementação específica de um método já definido em sua classe-base.
    - O método na classe derivada deve ter a mesma assinatura que o método na classe-base.
    - Usamos as palavras-chave virtual na classe-base para indicar que um método pode ser sobrescrito, e *override* na classe derivada para sobrescrever o método.

Programação C# – Polimorfismo

- Permite tratar objetos derivados como instâncias de sua classe-base, possibilitando a escrita de um código mais genérico e reutilizável.
- É expresso principalmente de duas maneiras: por meio da sobrecarga (*overloading*) de métodos e da substituição (*overriding*) de métodos. O polimorfismo de substituição (*overriding*) é uma forma.
- Quando uma classe derivada fornece uma nova implementação para um método que já existe em sua classe-base, dizemos que ela está sobrescrevendo esse método.
    - Isso permite que um objeto de uma classe derivada seja tratado como objeto de sua classe-base, mas ainda assim invoque o método apropriado da classe derivada de polimorfismo.

Interatividade

Qual das seguintes afirmações está correta sobre classes estáticas, interfaces e classes abstratas em C#?

a) Interfaces podem conter implementações de métodos e propriedades.
b) Classes estáticas podem ser instanciadas usando a palavra-chave *new*.
c) Classes abstratas são usadas para fornecer uma implementação parcial que deve ser completada por classes derivadas.
d) O modificador static é usado para declarar métodos que podem retornar diferentes tipos de objetos.
e) Uma classe abstrata pode conter apenas métodos abstratos e nenhuma implementação.

Resposta

Qual das seguintes afirmações está correta sobre classes estáticas, interfaces e classes abstratas em C#?

a) Interfaces podem conter implementações de métodos e propriedades.
b) Classes estáticas podem ser instanciadas usando a palavra-chave *new*.
c) Classes abstratas são usadas para fornecer uma implementação parcial que deve ser completada por classes derivadas.
d) O modificador static é usado para declarar métodos que podem retornar diferentes tipos de objetos.
e) Uma classe abstrata pode conter apenas métodos abstratos e nenhuma implementação.

Referências

- HUSKEY, V. R.; HUSKEY, H. D. Lady Lovelace and Charles Babbage. *Annals of the history of computing*, v. 2, p. 384, 1980.
- STEELE, G. L. Macaroni is better than spaghetti. In: *Proceedings of the 1977 symposium on artificial intelligence and programming languages*. Nova York: Association for Computing Machinery, 1977, p. 60-66.
- WILKES, M. V. The EDSAC – an electronic calculating machine. *Journal of Scientific Instruments*, p. 385-391, 1949.

ATÉ A PRÓXIMA!